Ford Car Reviews — Sentiment & Insight Extraction with LangChain

Class Exercise 1

Submitted by: Kushagra Gupta

Goal. From the first 25 Ford reviews, extract for each row: `Sentiment` (Positive / Neutral / Negative), `Pros`, `Cons`, `Liked_Features`, `Disliked_Features`.

Method (this notebook). I use a `PydanticOutputParser`: the schema's auto-generated format instructions are injected into the prompt, the LLM replies as JSON, and the parser validates it back into a typed object. Reviews are processed with LCEL `chain.batch(..., max_concurrency=2)` instead of a serial loop. Runs on **Groq** (free tier); concurrency is kept low to stay within free rate limits.

1. Setup

In [1]:
```
%pip install -q langchain langchain-core langchain-groq pandas pydantic
```

```
Note: you may need to restart the kernel to use updated packages.
d:\JIO_INSTITUTE\TERM 4\generative-ai-pgp-ji-2026\.venv\Scripts\python.exe: No modul
e named pip
```

In [2]:
```
import os, json, getpass
import pandas as pd
from langchain_groq import ChatGroq

# Groq free-tier key: https://console.groq.com/keys
if not os.environ.get("GROQ_API_KEY"):
    os.environ["GROQ_API_KEY"] = getpass.getpass("GROQ_API_KEY: ")

# qwen/qwen3.8-27b: Groq's current flagship, current as of this writing
# (llama-3.3-70b-versatile was deprecated by Groq on 2026-08-16).
# If you hit rate limits, switch to the lighter "openai/gpt-oss-20b".
model = ChatGroq(model="openai/gpt-oss-20b", temperature=0, max_retries=2)
print("Model:", model.model_name)
```

```
Model: openai/gpt-oss-20b
```

2. Read data — first 25 rows

`engine="python"` handles the newlines embedded in the review column.

In [3]:
```
raw = pd.read_csv("ford_car_reviews.csv", engine="python",
                  encoding="utf-8", on_bad_lines="skip")
```

```
reviews = raw.head(25).copy().reset_index(drop=True)
reviews = reviews.loc[:, ~reviews.columns.str.startswith("Unnamed")]
print(reviews.shape, "->", list(reviews.columns))
reviews[["Vehicle_Title", "Rating"]].head()
```

```
(25, 6) -> ['Review_Date', 'Author_Name', 'Vehicle_Title', 'Review_Title', 'Review',
'Rating']
```

Out[3]:

	Vehicle_Title	Rating
0	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	5.0
1	2006 Ford Mustang Coupe V6 Standard 2dr Coupe ...	3.0
2	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	5.0
3	2006 Ford Mustang Coupe V6 Deluxe 2dr Coupe (4...	5.0
4	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	5.0

3. Output schema + parser-based chain

The parser exposes `get_format_instructions()`, which we feed into the prompt via a partial variable. The chain is `prompt | model | parser`.

In [4]:

```
from typing import List
from pydantic import BaseModel, Field
from typing_extensions import Literal
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import PydanticOutputParser

class CarReviewInsight(BaseModel):
    sentiment: Literal["Positive", "Neutral", "Negative"] = Field(
        ..., description="Overall sentiment of the review.")
    pros: List[str] = Field(default_factory=list, description="Good points; [] if n
    cons: List[str] = Field(default_factory=list, description="Bad points; [] if no
    liked_features: List[str] = Field(default_factory=list,
        description="Specific features the reviewer liked.")
    disliked_features: List[str] = Field(default_factory=list,
        description="Specific features the reviewer disliked.")

parser = PydanticOutputParser(pydantic_object=CarReviewInsight)

template = PromptTemplate(
    template=(
        "You analyse customer car reviews. From the review below, determine the ove
        "sentiment and pull out pros, cons, and the specific vehicle features the r
        "liked or disliked (performance, comfort, price, mileage, styling, reliabil
        "Rely only on the text.\n\n"
        "Vehicle: {vehicle}\nReview: {review}\n\n{format_instructions}"
    ),
    input_variables=["vehicle", "review"],
    partial_variables={"format_instructions": parser.get_format_instructions()},
)
```

```
chain = template | model | parser
print("Chain built.")
```

```
Chain built.
```

Test on the first review

In [5]:
```
row0 = reviews.iloc[0]
out0 = chain.invoke({"vehicle": row0["Vehicle_Title"], "review": row0["Review"]})
print(out0.model_dump_json(indent=2))
```

```
{
  "sentiment": "Positive",
  "pros": [],
  "cons": [],
  "liked_features": [],
  "disliked_features": []
}
```

4. Batch-process all 25 reviews

`chain.batch` runs the calls concurrently. `return_exceptions=True` keeps a single failure from killing the batch; failed items become a `Neutral` placeholder.

In [6]:
```
inputs = [{"vehicle": r["Vehicle_Title"], "review": r["Review"]}
          for _, r in reviews.iterrows()]

raw_out = chain.batch(inputs, config={"max_concurrency": 2}, return_exceptions=True

parsed, failed = [], []
for i, o in enumerate(raw_out):
    if isinstance(o, Exception):
        failed.append((i, str(o)))
        parsed.append(CarReviewInsight(sentiment="Neutral"))
    else:
        parsed.append(o)

print(f"Processed {len(parsed)} reviews, {len(failed)} failed.")
for i, e in failed: print("  row", i, "->", e[:120])
```

```
Processed 25 reviews, 6 failed.
  row 9 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `op
enai/gpt-oss-20b` in organization `org_01kx3axh
  row 13 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `o
penai/gpt-oss-20b` in organization `org_01kx3axh
  row 16 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `o
penai/gpt-oss-20b` in organization `org_01kx3axh
  row 18 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `o
penai/gpt-oss-20b` in organization `org_01kx3axh
  row 20 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `o
penai/gpt-oss-20b` in organization `org_01kx3axh
  row 22 -> Error code: 429 - {'error': {'message': 'Rate limit reached for model `o
penai/gpt-oss-20b` in organization `org_01kx3axh
```

5. Attach results to the DataFrame

In [7]:
```
def as_text(xs):
    xs = [x.strip() for x in xs if x and x.strip()]
    return ", ".join(xs) if xs else "None mentioned"

reviews["Sentiment"]         = [p.sentiment for p in parsed]
reviews["Pros"]              = [as_text(p.pros) for p in parsed]
reviews["Cons"]              = [as_text(p.cons) for p in parsed]
reviews["Liked_Features"]    = [p.liked_features for p in parsed]
reviews["Disliked_Features"] = [p.disliked_features for p in parsed]

reviews[["Vehicle_Title", "Rating", "Sentiment", "Pros", "Cons",
         "Liked_Features", "Disliked_Features"]].head(25)
```

Out[7]:

	Vehicle_Title	Rating	Sentiment	Pros	Cons	Liked_Features	Disliked
0	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	5.000	Positive	None mentioned	None mentioned	[]	
1	2006 Ford Mustang Coupe V6 Standard 2dr Coupe ...	3.000	Positive	Engine is fine and sounds good, Great mileage,...	Orneriest transmission ever used, Difficult to...	[Engine performance, Fuel economy, Power deliv...	[Transm Rid R
2	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	5.000	Positive	fairly reasonable price, great investment	None mentioned	[price, investment value]	
3	2006 Ford Mustang Coupe V6 Deluxe 2dr Coupe (4...	5.000	Positive	High-performance modifications (air aid cold a...	None mentioned	[Air aid cold air injector, Throttle body spac...	
4	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	5.000	Positive	The car hugs the road and responds instantly t...	Alternator needed repair, Regular maintenance ...	[Road-hugging handling, Instant responsiveness...	[Altern
5	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	3.000	Negative	None mentioned	sensors issues, cam phasers, solenoid problems	[]	[se
6	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	4.625	Positive	Great bang for the buck performance, Decent fu...	None mentioned	[Performance, Fuel economy, Comfort, Ease of m...	
7	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	4.375	Positive	Upgraded Airaid air filter system, Flowmaster ...	None mentioned	[Airaid air filter system, Flowmaster exhaust,...	
8	2006 Ford Mustang Coupe GT Deluxe 2dr Coupe (4...	3.500	Negative	None mentioned	Engine starts having knocking noise and signif...	[]	pe Reliabi
9	2006 Ford Mustang	4.625	Neutral	None mentioned	None mentioned	[]	

	Vehicle_Title	Rating	Sentiment	Pros	Cons	Liked_Features	Disliked
	Coupe GT Premium 2dr Coupe (...						
10	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	4.500	Positive	fun driving, good handling, stylish appearance...	None mentioned	[fun driving, handling, styling]	
11	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	4.750	Positive	Fun to drive, Decent gas mileage, Well-maintai...	None mentioned	[Fun to drive, Decent gas mileage, Well-mainta...	
12	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	4.875	Positive	Reliable performance, No major issues, Low mai...	None mentioned	[Reliability, Low maintenance, Engine performa...	
13	2006 Ford Mustang Coupe V6 Standard 2dr Coupe ...	4.625	Neutral	None mentioned	None mentioned	[]	
14	2006 Ford Mustang Coupe V6 Deluxe 2dr Coupe (4...	4.375	Negative	drives great, no major issues beyond the liste...	dashboard rattles, dealer unable to fix the ra...	[overall driving performance, general reliabil...	[rat brake v
15	2006 Ford Mustang Coupe V6 Standard 2dr Coupe ...	4.375	Positive	Excellent handling on town, highway, and twist...	Transmission pan leak that required two repair...	[Handling, Reliability, Maintenance]	[Tra lea
16	2006 Ford Mustang Coupe V6 Standard 2dr Coupe ...	3.750	Neutral	None mentioned	None mentioned	[]	
17	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	4.000	Negative	Nice styling, Fun to drive	Unending mystery electrical problems, Dealersh...	[Styling, Driving experience]	Dealers
18	2006 Ford Mustang Coupe V6	4.750	Neutral	None mentioned	None mentioned	[]	

	Vehicle_Title	Rating	Sentiment	Pros	Cons	Liked_Features	Disliked
	Premium 2dr Coupe (...						
19	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	4.000	Neutral	Great looking car, Many options available (inc...	Very stiff ride, Lots of rattles, Alternator a...	[Styling, Option package selection]	[Ric (stiffnes
20	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	3.375	Neutral	None mentioned	None mentioned	[]	
21	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	4.875	Positive	drives like a dream, everything I ever imagine...	None mentioned	[performance, appearance, reliability]	
22	2006 Ford Mustang Coupe GT Premium 2dr Coupe (...	4.500	Neutral	None mentioned	None mentioned	[]	
23	2006 Ford Mustang Coupe V6 Premium 2dr Coupe (...	4.625	Positive	Great fuel economy, Good styling, Reliable per...	Noisy, Rough ride, Uncomfortable on long trips	[Fuel efficiency, Exterior styling, Reliability]	[Engi
24	2006 Ford Mustang Coupe GT Deluxe 2dr Coupe (4...	4.625	Positive	Fast performance, Great engine sound, Excellen...	None mentioned	[V8 engine, Cold air intake, Pypes Violator ex...	

Sample record (JSON)

In [8]:
```
rec = {k: reviews.loc[0, k] for k in
       ["Review_Date", "Vehicle_Title", "Rating", "Sentiment",
        "Pros", "Cons", "Liked_Features", "Disliked_Features"]}
rec["Review_Text"] = reviews.loc[0, "Review"][:80] + "..."
print(json.dumps(rec, indent=2, ensure_ascii=False, default=str))
```

```
{
  "Review_Date": " on 06/06/18 14:19 PM (PDT)",
  "Vehicle_Title": "2006 Ford Mustang Coupe GT Premium 2dr Coupe (4.6L 8cyl 5M)",
  "Rating": 5.0,
  "Sentiment": "Positive",
  "Pros": "None mentioned",
  "Cons": "None mentioned",
  "Liked_Features": [],
  "Disliked_Features": [],
  "Review_Text": " Doesn’t disappoint..."
}
```


6. Sanity check: does sentiment agree with the star rating?

A quick cross-tab of model sentiment against a rating bucket (low ≤2.5, mid ≤3.5, high >3.5) — a fast way to spot obvious misclassifications.

In [9]:
```
bucket = pd.cut(reviews["Rating"], bins=[-0.1, 2.5, 3.5, 5.1],
                labels=["low (<=2.5)", "mid (<=3.5)", "high (>3.5)"])
print("Sentiment counts:")
print(reviews["Sentiment"].value_counts().to_string())
print("\nSentiment vs rating bucket:")
print(pd.crosstab(reviews["Sentiment"], bucket).to_string())
```

```
Sentiment counts:
Sentiment
Positive    14
Neutral      7
Negative     4

Sentiment vs rating bucket:
Rating     mid (<=3.5)  high (>3.5)
Sentiment
Negative             2            2
Neutral              1            6
Positive             1           13
```

7. Save output

In [10]:
```
reviews.to_csv("ford_reviews_with_insights.csv", index=False)
print("Written: ford_reviews_with_insights.csv")
```

```
Written: ford_reviews_with_insights.csv
```

In [ ]: